PROJECT DOCUMENTATION

CO_2 Monitor

ESP32-C6 · Sensirion SCD-41 CO_2 sensor · SHT45 temperature & humidity · 0.96" OLED

Version 0.11 – Step 3: web page **Updated** 29 Oct 2026 **Status** breadboard prototype

1. Project overview
2. Hardware list
3. ESP32-C6 pin diagram
4. Wiring
5. Arduino IDE setup
6. Firmware
7. History storage
8. Web page
9. Testing
10. CO_2 calibration
11. Hardware notes
12. Troubleshooting
13. Case (design v3)
14. Change log

1 Project overview

Step 1 goal: the OLED shows the temperature and humidity from the **SHT45** and the CO_2 concentration from the **SCD-41**. To save power and prevent OLED burn-in, the display switches off after **30 seconds**. A **push button** wakes it for another 30 seconds. The sensors keep measuring while the display is off, and the dimmed **RGB LED** on the ESP32 board shows the CO_2 level all the time as a colour from green through yellow to red.

Step 2: the monitor joins your Wi-Fi, gets the time from the internet and keeps a **history** of the values in its own flash memory: 1-minute values for 7 days, 5-minute values for 90 days and hourly values for about 3½ years (section 7).

Step 3 (this version): the monitor serves its own **web page**. Any browser on the home network shows the live values, graphs of any stored day and time range, and a CSV download, in light or dark mode (section 8). Next step: Matter for the Alexa Echo.

The ESP32 talks to all three modules over one shared **I²C bus**, made of four wires: 3.3 V, GND, SDA (data) and SCL (clock). Each module has its own fixed address, so they don't get in each other's way. The button is a separate input on GPIO2.

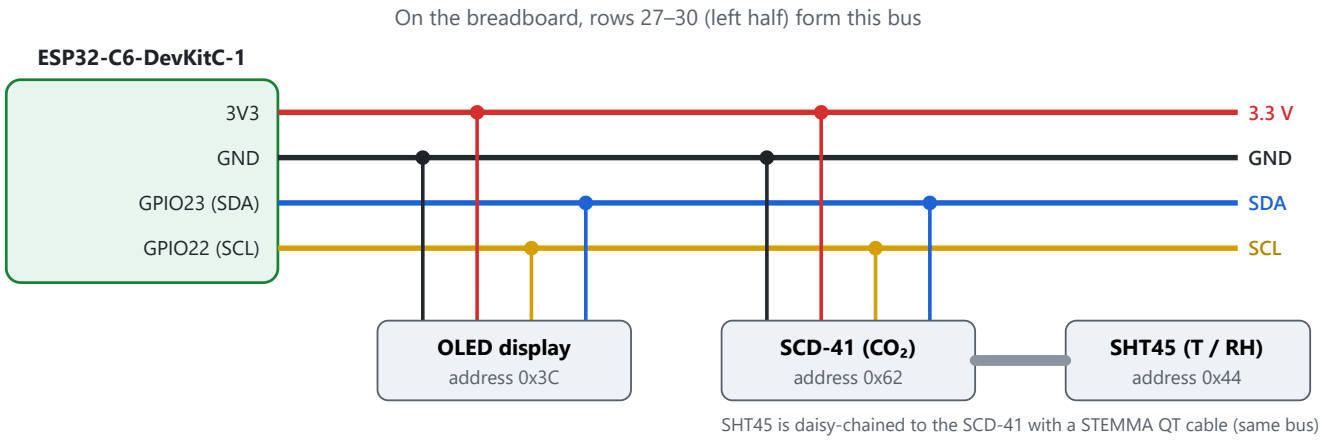


Figure 1 – Every module hangs on the same four wires. A dot means "connected"; lines that cross without a dot are not connected.

Project files

File	Purpose
`firmware/co2_monitor/co2_monitor.ino`	Main program (step 3): sensors → display, button wake, 30 s timeout, CO_2 colour on the RGB LED, serial output, Wi-Fi and internet time, Serial Monitor commands.
`firmware/co2_monitor/history.h`, `history.cpp`	History storage in the flash (section 7). The Arduino IDE shows them as tabs next to the main program.
`firmware/co2_monitor/web.h`, `web.cpp`	Web server: the page and the data addresses it uses (section 8).
`firmware/co2_monitor/web/index.html`, `make_web_page.ps1`, `../web_page.h`	The web page; the script packs it into `web_page.h`, the compressed copy the firmware serves (8.6).
`firmware/co2_monitor/secrets_example.h`	Template for `secrets.h`.
`firmware/co2_monitor/secrets.h`	Your Wi-Fi name and password (5.8). Not stored in git.
`firmware/i2c_scanner/i2c_scanner.ino`	Wiring check: lists all I^2C devices it finds. Run this first.
`firmware/scd41_calibration/scd41_calibration.ino`	Optional tool: forced recalibration (FRC) of the SCD-41 in fresh outdoor air.
`docs/CO2_Monitor_Documentation.html`	This document (source).
`docs/CO2_Monitor_Documentation.pdf`	PDF version, regenerated from the HTML after every change.
`docs/build_pdf.ps1`	Script that turns the HTML into the PDF (uses Chrome or Edge in headless mode).
`docs/web_page.png`	Screenshot of the web page (Figure 6).

2 Hardware list

#	Component	Role	Interface	Supply
1	**Espressif ESP32-C6-DevKitC-1-N8** ESP32-C6-WROOM-1-N8 module, 8 MB flash	Microcontroller. Reads the sensors, drives the display.	2× USB-C	5 V via USB → on-board 3.3 V regulator
2	**Adafruit SCD-41** (STEMMA QT) Sensirion SCD41	CO_2 sensor (photoacoustic NDIR)	I^2C `0x62`	3.3–5 V (we use 3.3 V)
3	**Adafruit SHT45** (STEMMA QT) Sensirion SHT45	Temperature and relative humidity	I^2C `0x44`	3–5 V (we use 3.3 V)
4	**Joy-it 0.96" OLED** 128×64 (SBC-OLED01) SSD1306 controller	Display	I^2C `0x3C` (or `0x3D`)	3.3–5 V (we use 3.3 V)
5	STEMMA QT → male header pins cable (4-pin)	Connects the breadboard bus to the SCD-41	JST SH 1.0 mm	–
6	STEMMA QT ↔ STEMMA QT cable	Daisy-chains SHT45 to the SCD-41	JST SH 1.0 mm	–
7	4-pin tactile push button	Wakes the display	GPIO2	–
8	400-point breadboard	Carries the ESP32, OLED, button and the bus	–	–

9	Male–male jumper wires, 5 (one of them long) (needed, not in the original list)	Breadboard connections	–	–
10	USB-C **data** cable (needed)	Power and programming. Charge-only cables don't work.	–	–

Planned: more hardware will be added in later steps. Add new I^2C devices to the same bus (check that their address is not already used); give anything else its own free GPIO from the pin diagram.

3 ESP32-C6-DevKitC-1 pin diagram

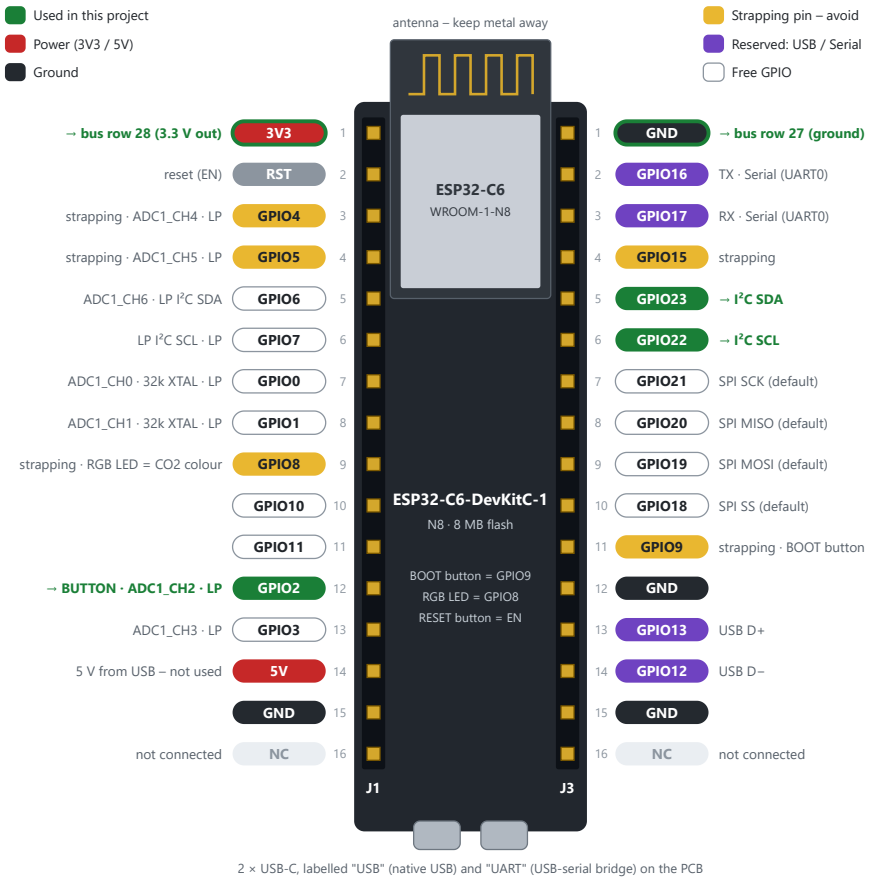


Figure 2 – Top view, antenna up, USB down. Green = used in this project. "LP" = low-power GPIO (can wake the chip from deep sleep). The drawing is schematic, not to scale; the labels printed on your board are what counts.

Pins used in step 1

Header pin	Signal	Used for	Breadboard hole → goes to
J1-1	**3V3**	3.3 V supply for OLED, SCD-41, SHT45	j19 → c28 (3.3 V bus row)
J1-12	**GPIO2**	Push button (internal pull-up, pressed = LOW)	j8 → j25 (button leg f25)
J3-5	**GPIO23**	I²C SDA (Arduino default for ESP32-C6)	a15 → c30 (SDA bus row)
J3-6	**GPIO22**	I²C SCL (Arduino default for ESP32-C6)	a14 → c29 (SCL bus row)
J3-1	**GND**	Ground	a19 → c27 (GND bus row)
(J1-9)	**GPIO8**	On-board RGB LED (CO_2 colour)	no wire, keep i11 empty

Pins to avoid (or use with care)

- **Strapping pins GPIO4, GPIO5, GPIO8, GPIO9, GPIO15:** their level during reset decides how the chip boots. A button or sensor pulling one of them low can stop the board from starting. GPIO8 also drives the on-board RGB LED, and GPIO9 is the BOOT button.
- **GPIO12 / GPIO13:** native USB (D− / D+). Using them breaks the "USB" port.
- **GPIO16 / GPIO17:** UART0 TX/RX, used by the "UART" USB port and the Serial Monitor.
- **5V pin:** carries 5 V from USB. **The GPIOs are not 5 V tolerant.** Never connect 5 V to a GPIO or to the breadboard bus.
- **Good free pins for later:** GPIO0, 1, 3, 6, 7 (low-power GPIOs, some with ADC), GPIO10, 11, 18, 19, 20, 21.

4 Wiring

Breadboard holes are named **column letter + row number**, e.g. `j19` = column j, row 19. On a 400-point breadboard, each row's five holes a–e are connected together, and so are f–j (but not across the centre gap). The long power rails run along the edges.

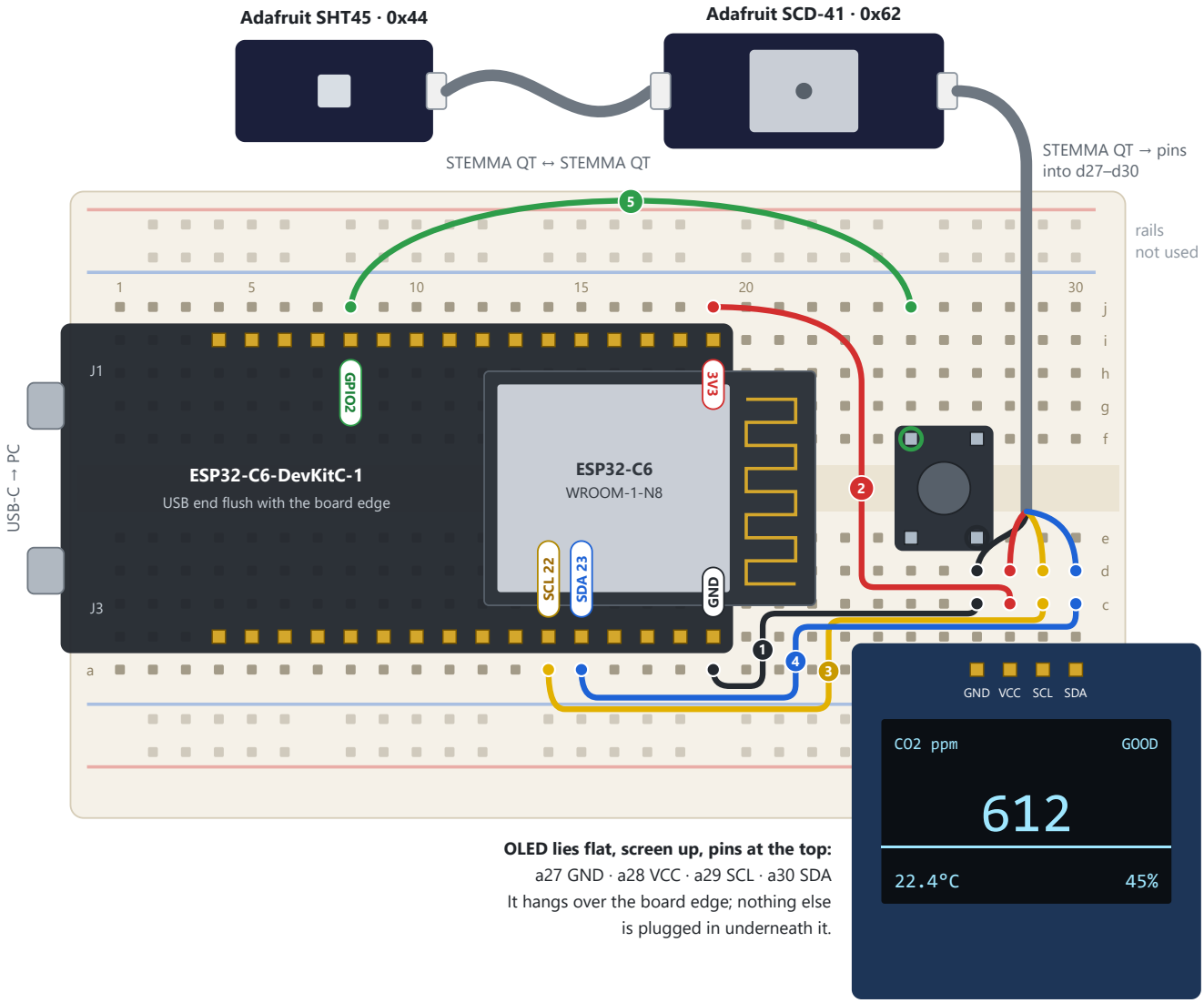


Figure 3 – Breadboard layout (top view, seen from the column-a side). The ESP32 sits in rows 4–19 (3V3 pin in i19) with its USB end flush with the board edge; the antenna overhangs rows 20–22. Rows 27–30, left half, form the bus in the same order as the OLED's pins: **row 27 = GND, row 28 = 3.3 V, row 29 = SCL, row 30 = SDA**. The OLED lies flat in a27–a30 and hangs over the board edge, the ESP32 wires go into column c, the SCD-41 cable into column d. The button's GND leg (e27) sits directly in the GND row. The power rails are not used.

Connection list

#	Wire	From	To	Signal
1	black	a19 (J3-1 GND)	c27	GND → bus row 27
2	red (long)	j19 (J1-1 3V3)	c28	3.3 V → bus row 28
3	yellow	a14 (J3-6 GPIO22)	c29	SCL → bus row 29
4	blue	a15 (J3-5 GPIO23)	c30	SDA → bus row 30
5	green	j8 (J1-12 GPIO2)	j25	Button, leg f25
–	OLED header	a27 GND · a28 VCC · a29 SCL · a30 SDA (lying flat, pins at the top)		plugs straight into the bus rows

–	STEMMA QT → pins	d27 black (GND) · d28 red (V+) · d29 yellow (SCL) · d30 blue (SDA); other end into the SCD-41	SCD-41 on the bus
–	STEMMA QT ↔ STEMMA QT	SCD-41 second connector → SHT45	SHT45 on the bus
–	Button	legs in e25, e27, f25, f27; leg e27 sits in the GND row, so it needs no wire of its own	GPIO2 ↔ GND

Each bus row holds four things: the OLED pin (column a), the ESP32 wire (column c), the SCD-41 cable pin (column d) and, in row 27, the button leg (column e). The power rails are not used in this layout.

Step by step (USB unplugged!)

1. **ESP32:** pins in rows 4–19, USB-C ports flush with the end of the breadboard, antenna overhanging rows 20–22, the pin labelled **3V3** in **i19**. One free hole is left next to every pin: column **j** on the J1 side, column **a** on the J3 side.
2. **OLED:** plug its header into **a27–a30** so that you can read the screen with the pins at the top: **GND → a27, VCC → a28, SCL → a29, SDA → a30**. The module lies flat, screen up, and hangs over the edge of the breadboard. Nothing else is plugged in underneath it.
3. **ESP32 to the bus:** wire 1 (a19 → c27, GND), wire 3 (a14 → c29, SCL), wire 4 (a15 → c30, SDA). Then wire 2 (j19 → c28, 3.3 V); it is the long one that crosses the board just past the antenna end.
4. **SCD-41:** plug the STEMMA QT → pins cable into one of its connectors. Put the pins in d27 (black), d28 (red), d29 (yellow), d30 (blue): the same order as the OLED.
5. **SHT45:** connect it to the SCD-41's second connector with the STEMMA QT ↔ STEMMA QT cable. Place it as far from the ESP32 and the SCD-41 as the cable allows.
6. **Button:** place it across the centre gap in rows 25–27, legs in e25, e27, f25, f27. Then wire 5 (j8 → j25). The button's leg e27 is in the GND row and its diagonally opposite leg f25 is wired to GPIO2. Diagonal legs work whichever way round the button's internal leg pairs are.
7. **Check before plugging in USB:** row 27 (GND) and row 28 (3.3 V) are not connected to each other; nothing is connected to the 5V pin (J1-14, hole i6); the STEMMA colours are in the right rows.

No pins pointing out of the back of your OLED, or not enough room? Connect it with four female–male jumper wires to the same rows instead (GND → row 27, VCC → 28, SCL → 29, SDA → 30) and put it wherever you like.

> **Power the sensors from 3.3 V only.** The Adafruit boards would accept 5 V, but their I²C pull-up resistors connect to their supply pin. At 5 V they would pull SDA/SCL to 5 V, and the ESP32-C6 pins are not 5 V tolerant.

5 Arduino IDE setup

> **Already done on this PC:** Arduino IDE 2 is installed, and so is the board package **esp32 by Espressif Systems 3.3.12**. You still need to install the libraries (5.3) and the CP210x USB driver for the "UART" port (5.5).

5.1 Arduino IDE

Download **Arduino IDE 2.x** from `arduino.cc/en/software` and install it with the default options. Allow the USB driver installation if Windows asks.

5.2 ESP32 board package

1. **Tools → Board → Boards Manager…**, search `esp32`.

2. Install **"esp32 by Espressif Systems"**, version 3.x. (Not "Arduino ESP32 Boards", which is a different package for Arduino's own boards.)
3. If it isn't listed: **File → Preferences → Additional boards manager URLs**, add `https://espressif.github.io/arduino-esp32/package_esp32_index.json`, then search again.

5.3 Libraries

Sketch → Include Library → Manage Libraries... (or the books icon on the left). For each library, click **INSTALL**. When the IDE asks about dependencies, choose **INSTALL ALL**.

Search for	Install	Pulls in automatically	Tested version
`Adafruit SSD1306`	Adafruit SSD1306 (by Adafruit)	Adafruit GFX Library, Adafruit BusIO	2.5.17 / GFX 1.13.0 / BusIO 1.17.4
`Adafruit SHT4x`	Adafruit SHT4x Library (by Adafruit)	Adafruit Unified Sensor (+ Adafruit SH110X, unused)	1.0.5 / Unified Sensor 1.1.15
`Sensirion I2C SCD4x`	Sensirion I2C SCD4x (by Sensirion)	Sensirion Core	1.1.0 / Core 0.7.3

> Use **Sensirion I2C SCD4x version 1.x**. Version 0.4 and older have a different API (class `SensirionI2CScd4x`), and the sketches won't compile with it.

5.4 Board settings (Tools menu)

Setting	Value
Board	**ESP32C6 Dev Module** (Tools → Board → esp32)
Port	the COM port that appears when you plug in the board
USB CDC On Boot	**Disabled** if you use the port labelled **UART** · **Enabled** if you use the port labelled **USB**
Flash Size	**8MB (64Mb)**
Partition Scheme	**8M with spiffs (3MB APP/1.5MB SPIFFS)**. The history lives in its 1.5 MB data area (section 7).
CPU Frequency	160MHz (WiFi)
Upload Speed	921600 (use 115200 if uploads fail)
Erase All Flash Before Sketch Upload	**Disabled** (the default). Enabled deletes the stored history.
Everything else	leave at default

> **Keep Flash Size and Partition Scheme as shown.** Normal uploads keep the stored history. Changing either setting moves the data area, so the history can't be read any more. The firmware checks the layout at every start: with a different scheme it prints `History OFF: wrong partition scheme` and records nothing.

5.5 Which USB port?

Both USB-C ports can power and program the board. **Recommended: the port labelled "UART"** with *USB CDC On Boot = Disabled*. Serial output then appears from the first moment after reset. If you use the "USB" port, set *USB CDC On Boot = Enabled*, otherwise the Serial Monitor stays empty.

Drivers: the "USB" port uses a driver built into Windows and works straight away. The "UART" port goes through a **Silicon Labs CP2102N** USB-to-UART chip, and **on this PC Windows had no driver for it**. Device Manager listed

"CP2102N USB to UART Bridge Controller" under *Other devices* with error *Code 28*, and no COM port appeared. Windows Update doesn't install this driver automatically for this chip, so install it once by hand:

1. Download **CP210x Universal Windows Driver** (`CP210x_Universal_Windows_Driver.zip`) from `silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers`, tab *Downloads*.
2. Extract the ZIP completely. Installing from inside the ZIP fails.
3. Device Manager → right-click "CP2102N USB to UART Bridge Controller" → **Update driver** → **Browse my computer for drivers** → select the extracted folder → **Next**. Alternatively, right-click `silabser.inf` → *Show more options* → **Install**.
4. The device moves to *Ports (COM & LPT)* as **"Silicon Labs CP210x USB to UART Bridge (COM*n*)"**. Select that COM*n* in the Arduino IDE. If it still shows the warning, unplug the board, plug it back in, or restart the PC.

If no device appears at all, not even under *Other devices*, try another cable: many USB-C cables are charge-only.

5.6 First upload test

1. Plug in the board, select board and port as above.
2. **File** → **Examples** → **ESP32** → **GPIO** → **BlinkRGB** and click **Upload** (→ arrow).
3. The on-board RGB LED should cycle through colours. The toolchain works.
4. If the upload says "*Failed to connect*": hold `BOOT`, tap `RESET`, release `BOOT`, then upload again. Press `RESET` after the upload finishes.

5.7 Open the project sketches

File → **Open...** and pick the `.ino` file inside `firmware/<name>/`. The Arduino IDE needs each sketch in a folder with the same name, which is how the project is laid out. Serial Monitor: **Tools** → **Serial Monitor**, set to **115200 baud**.

5.8 Wi-Fi name and password (`secrets.h`)

The monitor needs Wi-Fi for the clock, and later for the web page. Your network name and password go in `firmware/co2_monitor/secrets.h`. Git ignores this file, so the password never ends up in the repository. The IDE shows it as a tab next to `co2_monitor.ino`.

1. If `secrets.h` is missing (for example in a fresh copy of the project), copy `secrets_example.h` and rename the copy to `secrets.h`. Without it the sketch doesn't compile.
2. Replace `your-wifi-name` and `your-wifi-password`. Keep the quotes.
3. The ESP32-C6 only uses **2.4 GHz** Wi-Fi. A router that sends 2.4 and 5 GHz under one name is fine.

```
constexpr char WIFI_SSID[] = "MyNetwork";
constexpr char WIFI_PASSWORD[] = "my secret password";
```

The monitor shows up in your router's device list as **co2-monitor**. For the web page, reserve a fixed address for it in the router later (DHCP reservation).

6 Firmware (`co2_monitor.ino`)

What it does

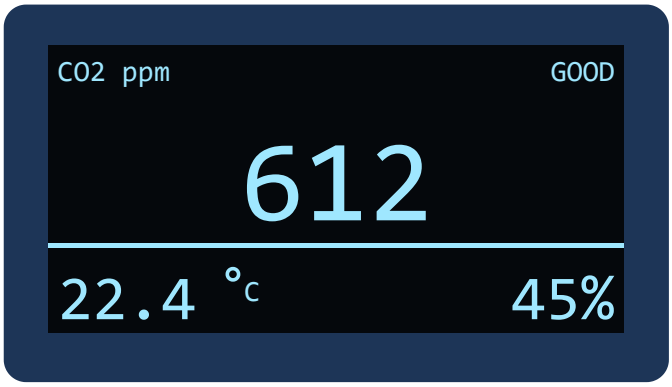


Figure 4 – Display layout: CO_2 and rating on top, temperature and humidity (from the SHT45) at the bottom.

- Starts the I^2C bus on GPIO23 (SDA) / GPIO22 (SCL).
- **SHT45**: high-precision reading every 2 s, heater off.
- **SCD-41**: periodic measurement, a new CO_2 value every 5 s. Altitude compensation is set at every start.
- **Warm-up**: for the first 30 s after start-up the sensors measure, but no value is shown, printed or saved. The values from this time are thrown away. The display counts down ("warming up", 30 s … 1 s).
- **Display**: on during the warm-up, then for 30 s with the first values, and for 30 s after every button press. Pressing while it's on restarts the 30 s. When off, the panel is fully dark and the content is kept. Brightness is set by `OLED_BRIGHTNESS`.
- **RGB LED** (on the board, GPIO8): dimmed, always on, shows the CO_2 level as a colour that changes smoothly from green through yellow to red. It stays off until the first CO_2 value after the warm-up.
- **Serial**: one line per CO_2 reading after the warm-up, usable in the Serial Monitor and the Serial Plotter.
- **Wi-Fi and clock**: connects with the details in `secrets.h` and reconnects by itself. Gets the time (UTC) from the internet and updates it every 3 hours.
- **History**: stores 1-minute, 5-minute and hourly values in the flash (section 7). You can view them with commands in the Serial Monitor (7.6).
- **Web page**: live values, graphs and CSV downloads in a browser on the home network (section 8).
- A missing module is reported on Serial and on the display (`--.-` / "SCD-41 missing"). The rest keeps working.

Settings at the top of the sketch

Constant	Default	Meaning
`SENSOR_ALTITUDE_M`	0	**Set this to your altitude in metres above sea level.** The SCD-41 uses it for air-pressure compensation. Without it, readings at e.g. 500 m are a few percent off.
`WARMUP_TIME_MS`	30000	Warm-up after start-up (ms): the sensors measure, but nothing is shown, printed or saved.
`OLED_ADDRESS`	0x3C	0x3D if the address resistor on the back of the OLED was moved.
`OLED_ROTATION`	0	0 = normal, header pins at the top. Set 2 if the text appears upside down (e.g. if you mount the display the other way round).

`OLED_BRIGHTNESS`	207	Display brightness 0–255 (the SSD1306's contrast setting; 207 is the library default). The range is smaller than it sounds: even 0 stays readable, at roughly half brightness. Lower values also slow burn-in.
`DISPLAY_ON_TIME_MS`	30000	How long the display stays on (ms).
`BUTTON_PIN`	2	Button GPIO (other leg to GND).
`I2C_SDA_PIN` / `I2C_SCL_PIN`	23 / 22	I²C pins.
`CO2_GOOD_BELOW`, `CO2_OK_BELOW`, `CO2_VENTILATE_BELOW`	800 / 1000 / 2000	Limits for the rating shown at the top right, and the points where the LED is green, yellow and red.
`LED_BRIGHTNESS`	5	Brightness of the RGB LED, 1–255 (0 = off). 255 is glaring; the BlinkRGB example uses 64. Try 3–5 for a bedroom, 20–30 for a bright room.
`LED_GREEN`, `LED_YELLOW`, `LED_RED`	{0,255,0} / {255,160,0} / {255,0,0}	The three colours at full brightness, as {red, green, blue}. Yellow has less green because the green LED looks brighter than the red one. If yellow looks too green, lower its middle value.
`WIFI_HOSTNAME`	co2-monitor	The name shown in your router's device list.
`NTP_SERVER_1`, `NTP_SERVER_2`	pool.ntp.org / time.google.com	Internet time servers. Your router's address also works if it offers time itself.

The Wi-Fi name and password are in `secrets.h` (5.8). The history settings are in `history.cpp` (7.9).

CO_2 rating on the display and the LED

CO_2 (ppm)	Display	LED	Meaning
below 800	**GOOD**	green	Well-ventilated room (outdoor air is about 420–430 ppm).
800 – 999	**OK**	green → yellow	Acceptable.
1000 – 1999	**VENTILATE** (inverted)	yellow → orange → red	Air is getting stale. Open a window. 1000 ppm is the classic guideline (Pettenkofer value).
2000 and above	**BAD AIR** (inverted)	red	Ventilate now. Tiredness and poor concentration are common at these levels.

The LED colour changes smoothly with the CO_2 value; for example, 1500 ppm is orange. It is updated with every new reading (every 5 s) and stays on while the display is off.

7 History storage

The monitor keeps a long-term record of its values in the ESP32's own flash memory, with no extra hardware. The web page (section 8) draws its graphs from it and offers it as a CSV download. You can also read it in the Serial Monitor (7.6). The code is in `history.h` and `history.cpp`.

7.1 Three levels of detail

Level	Each record holds	Record	Files	Kept
1-minute (`m1`)	average CO_2, temperature, humidity and air pressure	12 bytes	`/m1/20261028.bin`, one per day, up to 17 KB	7 days

5-minute (`m5`)	the same	12 bytes	`/m5/20261028.bin`, one per day, up to 3.5 KB	90 days
Hourly (`h1`)	average, minimum and maximum of CO_2, temperature and humidity; average pressure; number of minutes measured	26 bytes	`/h1/202610.bin`, one per month, up to 19 KB	until the flash is full, about 3½ years; then the oldest month is deleted

- Air pressure stays empty until the BMP390 is fitted. The light level isn't stored; the VEML7700 is meant for the display brightness.
- The hourly minimum and maximum come from the 1-minute averages, not from single 5-second readings.
- Each file starts with a format version, so a later firmware can add values without losing the older data.

7.2 How a reading gets saved

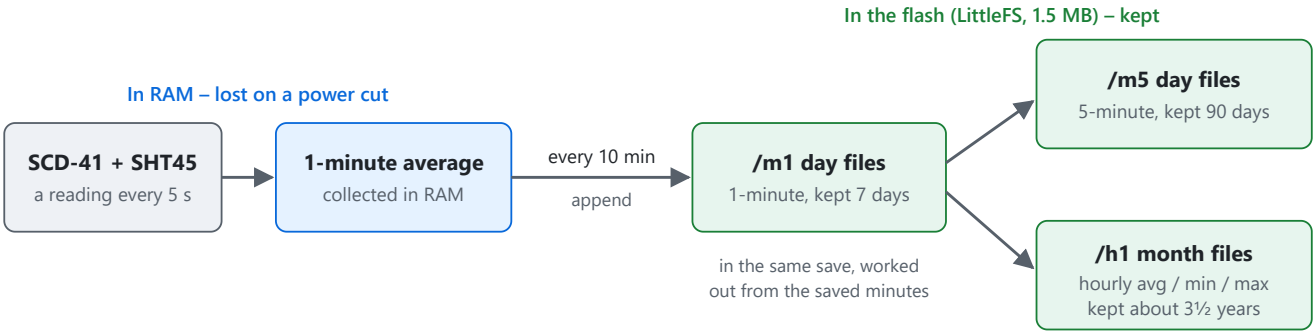


Figure 5 – How a reading becomes history. Only the part in RAM, at most 10 minutes, is lost on a power cut.

1. Every 5 s the SCD-41 delivers a CO_2 value. Together with the latest SHT45 values it is added to the average of the current minute, in RAM. Nothing is stored during the 30 s warm-up after a start.
2. Every 10 minutes (at :00, :10, :20 … plus 5 s) the finished minutes are appended to today's file in `/m1`.
3. In the same save, every finished 5-minute period and every finished hour is worked out from the saved 1-minute values and appended to `/m5` and `/h1`. Because they always come from saved data, a power cut doesn't leave a hole in them: the hour that was running is completed after the restart from what was saved before.
4. Files older than their retention are deleted. If less than 64 KB is free, the oldest hourly month is deleted.

Records are only ever added to the end of a file, never changed later. Flash memory can't change data in place, so editing old records would wear it out much faster.

7.3 Time

- The clock comes from the internet over Wi-Fi. All values are stored in **UTC**; the web page converts them to your local time, summer time included. That's why a day file starts at midnight UTC, not at local midnight.
- If the internet is down after a restart, the monitor doesn't know the time yet (it has no clock battery). It keeps up to **24 hours** of 1-minute values in RAM and gives them their correct times as soon as the clock is set. If the clock still isn't set after that, the oldest minutes are dropped.
- Without Wi-Fi details in `secrets.h` nothing is recorded, because the values would have no time.

7.4 Power cuts, moving and flash wear

- **Power cut:** the saved history is safe. LittleFS finishes a save completely or not at all, so files are never damaged. You lose at most the last 10 minutes. A firmware upload or the `RESET` button counts as a power cut too. Before you unplug or upload on purpose, type `save` in the Serial Monitor.
- **Moving the monitor:** the same as a power cut. The history stays on the device. In another home, only `secrets.h` needs the new Wi-Fi.

- **Flash wear:** flash wears when a 4 KB block is erased, and each block is rated for at least 100,000 erases. A save erases about 2–3 blocks, and LittleFS spreads the erases over the roughly 150 blocks that are reused again and again. That's about 2 erases per block per day, or around 100 years until the rating is reached.

7.5 What erases the history

> **Normal firmware uploads keep the history,** including uploads of the scanner and calibration sketches. It is lost only through:
>
> - a different **Flash Size** or **Partition Scheme** in the Tools menu (5.4). The firmware notices this at start, prints `History OFF: wrong partition scheme` and doesn't touch the flash. Set the menu back and upload again; the history is normally still there.
> - **Erase All Flash Before Sketch Upload: Enabled** in the Tools menu. Leave it Disabled.
>
> If you ever need either of them, save a CSV copy first (7.6).

7.6 Serial Monitor commands

Type a command in the Serial Monitor's input line and press `Enter`. The line-ending box next to it must not be set to "No Line Ending".

Command	What it shows or does
`status`	Wi-Fi, clock, flash use, minutes not saved yet, and the files of each level with their newest value.
`files`	Every history file with its size and number of values.
`csv m1 1`	The 1-minute values of the last day as CSV. `m5` = 5-minute, `h1` = hourly; the number is how many days back (`csv h1 400` = more than a year of hourly values).
`save`	Saves the minutes waiting in RAM now, for example before you unplug the monitor.

Example answer to `status`:

```
Wi-Fi: connected to "MyNetwork", address 192.168.1.50, signal -58 dBm
Clock: 2026-10-28 17:21:40 UTC
Flash: 32 KB of 1536 KB used (2%)
Not saved yet (in RAM): 2 minutes, next save in 505 s
1-minute values (m1): 1 file, 20261028.bin ... 20261028.bin, newest value 2026-10-28 17:19 UTC
5-minute values (m5): 1 file, 20261028.bin ... 20261028.bin, newest value 2026-10-28 17:15 UTC
Hourly values (h1): 0 files
```

And the start of `csv m1 1`:

```
time_utc,co2_ppm,temperature_c,humidity_pct,pressure_hpa
2026-10-28 17:05,612,22.41,45.30,
2026-10-28 17:06,618,22.43,45.12,
```

To keep a copy, select the lines in the Serial Monitor, paste them into a text file and save it as `.csv`. If Excel puts everything in one column, open it with *Data → From Text/CSV* and choose comma as the separator. Times are UTC. An empty field means the value wasn't measured. The web page's download (8.4) is easier: it uses local time and Excel's number format.

7.7 File format

Each file is a 16-byte header followed by fixed-size records in time order. Numbers are little-endian, and values are stored as whole numbers:

Field	Bytes	Unit	Means "missing"
Header (16 bytes): the letters `CO2H`, format version (1), minutes per record (1, 5 or 60), record size, creation time, 4 reserved bytes.			
1-minute and 5-minute record (12 bytes):			
time	4	UTC seconds since 1970, start of the minute or 5 minutes	–
CO_2	2	ppm	0
temperature	2 (signed)	0.01 °C	−32768
humidity	2	0.01 %RH	65535
air pressure	2	0.1 hPa	0
Hourly record (26 bytes): time (4), CO_2 average, minimum and maximum (3 × 2), the same for temperature and for humidity, average air pressure (2), number of minutes measured (1; 60 = complete hour), 1 reserved byte. Same units and "missing" values as above.			

7.8 How much fits

Part of the 1.5 MB	Space
1-minute files: 8 days of 17 KB (5 blocks of 4 KB each)	160 KB
5-minute files: 91 days of 3.5 KB (1 block each)	364 KB
LittleFS's own data, plus 64 KB that is always kept free	about 110 KB
Left for hourly values, at 5 blocks (20 KB) per month or 240 KB per year	**about 900 KB, roughly 3½ years**

LittleFS stores each file in whole 4 KB blocks, so space is counted in blocks. Keeping 5-minute values for 30 instead of 90 days (7.9) leaves room for about 4½ years of hourly values. A microSD card would allow much more.

7.9 Settings in `history.cpp`

Constant	Default	Meaning
`MINUTE_KEEP_DAYS`	7	Days of 1-minute values.
`FIVE_MINUTE_KEEP_DAYS`	90	Days of 5-minute values.
`SAVE_INTERVAL_S`	600	Seconds between saves, and the most a power cut can lose. Shorter means more flash wear.
`PENDING_MAX`	1440	Minutes kept in RAM while the clock isn't set (24 h, 17 KB of RAM).
`MIN_FREE_BYTES`	64 KB	When less is free, the oldest hourly month is deleted.

8 Web page

The monitor serves its own web page on your home network. It shows the live values, graphs of any stored day and a CSV download. It needs no app and no internet: any browser on a phone, tablet or PC in the same Wi-Fi works. The code is in `web.h` and `web.cpp`, the page in `web/index.html`.

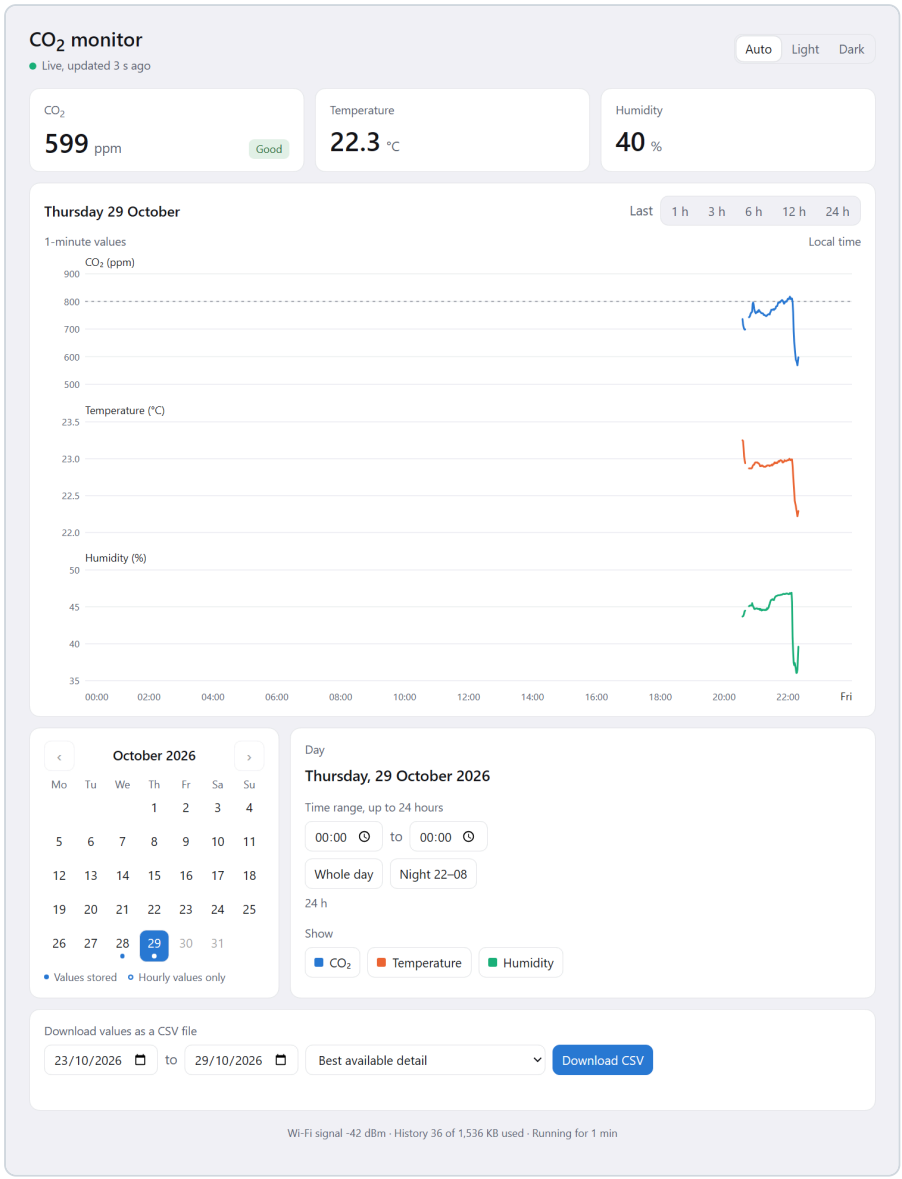


Figure 6 – The web page in light mode, on the first evening with stored values (29 Oct 2026). The short break at about 21:40 is a restart for a firmware upload.

8.1 Opening the page

- At the start the Serial Monitor shows `Wi-Fi connected, web page at http://192.168.178.40/`, and `status` shows it too. Type that address into the browser and save it as a bookmark, or on the phone's home screen.
- The phone or PC must be in the **same Wi-Fi**. A guest network usually can't reach devices in the home network.
- There's no password. Anyone in your Wi-Fi can look at the values, but nothing can be changed from the page.

8.2 Fixed address in the router

The router gives every device its address. It normally keeps it, but after a router reset or a long time switched off the monitor may get another one, and the bookmark stops working. Tell the router to always give the monitor the same address. The monitor is listed there as **co2-monitor**.

- **FRITZ!Box** (home addresses 192.168.178.x): *Heimnetz → Netzwerk → Netzwerkverbindungen*, pencil next to co2-monitor, tick *Diesem Netzwerkgerät immer die gleiche IPv4-Adresse zuweisen*, then OK. In English: *Home Network → Network → Network Connections*, *Always assign this network device the same IPv4 address*. The menu names differ a little between FRITZ!OS versions.
- **Other routers:** look for "DHCP reservation", "static lease" or "address reservation" in the LAN or DHCP settings.

> The address isn't fixed in the firmware on purpose. With a new router or in another network the monitor would then not get online at all, so it would get no time and store nothing. A name like `co2-monitor.local` isn't used either, because the Matter step brings its own name service and the two would clash.

8.3 What the page shows

Part	What it does
Status line	Green dot: a new reading arrived within the last 15 s (the page asks every 5 s). Red dot: no connection. "Waiting for the internet time" while the monitor's clock isn't set.
Live values	CO_2 with the same rating as the display (Good, OK, Ventilate, Bad air; limits in section 6), temperature and humidity. During the 30 s warm-up: "Warming up" with the seconds left.
Graphs	Right below the live values. One graph per value, all on the same time axis in local time; the heading above them names the time shown. Move the mouse over them, or drag a finger along them, to see the values at that moment. Dashed lines mark the CO_2 limits that fall inside the scale. A gap means nothing was measured (power off, restart, warm-up). While the range includes the current time, new values are added every minute.
Last 1–24 h	The buttons *1 h*, *3 h*, *6 h*, *12 h* and *24 h* above the graphs show the last hours up to now. The range then moves along with the time. Picking a day or changing a time ends it; a day picked after it is shown whole.
Detail	The last 7 days use the 1-minute values, up to 90 days the 5-minute values, older days the hourly averages with a shaded band from the hourly minimum to the maximum. The line above the graphs says which.
Day and calendar	Below the graphs: the chosen day and time range on the right, the calendar on the left (on a phone the day box comes first). In the calendar, a dot under a day means values are stored for it, a ring that only hourly values are left (days older than 90 days). Future days can't be chosen. The arrows change the month.
Time range	From and to, up to 24 hours. If "to" is earlier than "from", the range ends the next day. *Whole day* and *Night 22–08* set it with one click.
Show	Switches the CO_2, temperature and humidity graphs on and off. The browser remembers the choice.
Auto / Light / Dark	Auto follows the light or dark setting of the phone or PC. Light and Dark override it. The browser remembers the choice.
Bottom line	Wi-Fi signal, flash used by the history, and the time since the monitor started.

8.4 CSV download

Choose a start and an end date (both included) and the detail, then press *Download CSV*. The browser fetches the values from the monitor in parts and saves them as a file, for example `co2_2026-10-23_2026-10-29_1min.csv`.

- **Best available detail:** 1-minute values if the start date is within the last 7 days, 5-minute values within the last 90 days, otherwise hourly values. One file always has one level of detail.
- **Columns:** `time_local` (your time, summer time included), `time_utc`, then CO_2 in ppm, temperature in °C and humidity in %. Hourly files have the average, minimum and maximum of each, plus `minutes` (1-minute values in

that hour; 60 = complete). Each time is the start of its minute, 5 minutes or hour. A pressure column appears once the BMP390 is added. An empty field means the value wasn't measured.

- **Excel:** the file uses the browser's number format. With German settings that's `22,5` with `;` between the columns, so Excel opens it straight into columns.

8.5 What the web page costs

Resource	Use
Program flash	About 60 KB more: 1,166,586 of 3,342,336 bytes (34%), up from 1,106,244. About 41 KB is the web server library and 12 KB the page (35 KB before compression). Plenty is left for Matter.
RAM	0.4 KB more fixed RAM, plus a few KB while a request is answered.
History storage	**Unchanged.** The page is part of the firmware, not of the 1.5 MB data partition, and reading the history doesn't wear the flash. The keeping times in 7.1 stay the same.
Measuring	Every answer is below 60 KB and takes well under a second. The main program waits meanwhile, but no reading is lost: the SCD-41 keeps its newest value until it is read.

8.6 Changing the page

- The page is `web/index.html`: HTML, CSS and JavaScript in one file, with no outside libraries. `web_page.h` holds a compressed copy that the firmware serves. After changing the page, run `web\make_web_page.ps1` in PowerShell, then upload the firmware.
- To try a change without uploading, open the file in Chrome or Edge with the monitor's address added: `…/web/index.html?device=192.168.178.40`. It then loads its data from the monitor.

The page gets its data from these addresses, which also work in your own scripts. Times are UTC seconds since 1970.

Address	Answer
`/api/now`	Live values, warm-up seconds, CO_2 limits, Wi-Fi signal and clock state (JSON).
`/api/info`	History on or off, flash use, minutes not saved yet, and the oldest file and days kept for each level (JSON).
`/api/days?from=…&to=…`	The UTC days with values and their finest detail, for the calendar marks (JSON, at most 45 days).
`/api/data?res=1|5|60&from=…&to=…`	The stored values as binary records, exactly as in the files (7.7). At most 2 days of 1-minute, 14 days of 5-minute or 92 days of hourly values per request.

9 Testing

Test in this order. Each step builds on the one before, so when something fails you know where to look.

Step 1 – Power-on check

Plug in USB. The ESP32's power LED lights up, and so do the green power LEDs on the Adafruit boards (if present). Nothing gets warm. If anything gets hot, unplug immediately and check for a short between + and −.

Step 2 – I²C scan (`i2c_scanner.ino`)

Upload the scanner and open the Serial Monitor (115200). Every 5 s you should see:

```
Scanning I2C bus (SDA = GPIO23, SCL = GPIO22) ...
  0x3C  OLED display (SSD1306)
  0x44  SHT45 temperature / humidity
  0x62  SCD-41 CO2
3 device(s) found. Expected: 0x3C (or 0x3D), 0x44, 0x62
```

- **Nothing found:** check wires 3 and 4 (SDA/SCL swapped?) and the power wires 1 and 2.
- **Only one module missing:** check that module's four connections (OLED pin order! STEMMA cable fully seated?).
- **SHT45 missing but SCD-41 found:** check the STEMMA↔STEMMA cable between them.

Step 3 – Main program (`co2_monitor.ino`)

Set `SENSOR_ALTITUDE_M`, upload, open the Serial Monitor. Expected output:

```
CO2 monitor starting
SHT45 found
SCD-41 found, serial 0x...
SCD-41 automatic self-calibration (ASC): enabled
SCD-41 measuring
Warm-up: the first values follow in 30 s
Warm-up done, values are shown and saved from now on
CO2_ppm:612 Temperature_C:22.41 Humidity_pct:45.3
CO2_ppm:615 Temperature_C:22.42 Humidity_pct:45.2
```

The display shows "starting…", then "warming up" with a countdown from 30 s and `--.-` / `--%` at the bottom. After the countdown the temperature and humidity appear at once, and CO_2 within 5 s ("measuring…" until then). The RGB LED on the board goes off at the start and lights up dimly in the CO_2 colour with the first value. In normal room air it is green or yellowish. If the text is upside down, change `OLED_ROTATION` (2 ↔ 0) and upload again. **Serial Plotter** (Tools → Serial Plotter, 115200) draws the values as a graph. Untick Temperature and Humidity to see the CO_2 curve at a useful scale.

Step 4 – Button and display timeout

1. About 30 s after the warm-up ends (1 minute after start-up) the display goes dark. Serial output continues, so the sensors are still running.
2. Press the button. The display turns on immediately with current values.
3. Press again after ~20 s. The 30 s start over.

Step 5 – Sensor plausibility

- **Warm-up:** let everything run for **5–10 minutes** before judging the values. The SCD-41 needs a few minutes to settle, and the SHT45 needs time to reach room temperature after you handled it.
- **CO_2 breath test:** breathe gently toward the SCD-41 from 20–30 cm for a few seconds. Within 10–30 s the value jumps to well over 1000 ppm (several thousand is normal), then falls back over a few minutes. The LED goes through yellow and orange to red and back. This proves the sensor reacts. Don't breathe directly onto it from close up again and again; the moisture doesn't help it.
- **Humidity:** breathe near the SHT45. RH jumps by several percent and recovers within a minute.
- **Temperature:** hold a finger near (not on) the SHT45. Temperature rises slowly. Compare with another thermometer once it has settled; the SHT45 (±0.1 °C) is usually the more accurate one.

Step 6 – Fresh-air reference check

After the sensor has run for at least 10 minutes, take the setup **outdoors** (shade, not in direct wind, nobody breathing near it) on a power bank or laptop. Or air the room thoroughly: windows wide open, ideally a cross-draught, for 10–15 minutes. Outdoors it should read about **400–480 ppm**.

- Expected accuracy: ±(40 ppm + 5 % of reading), roughly ±60 ppm at 425 ppm. Readings between about 370 and 490 ppm outdoors are fine.
- A ventilated room rarely reaches true outdoor levels. 450–550 ppm indoors after airing is normal.
- Clearly off outdoors (e.g. above 550 or below 350 ppm): see Calibration (FRC).

Step 7 – Long run

Leave it running for at least 24 h, ideally a week. Typical pattern: CO_2 rises while people are in the room with the door and windows closed (a bedroom can reach 1500–2500 ppm overnight), and drops quickly when you air the room. If values drift or seem stuck, check the Serial output for errors.

Step 8 – Wi-Fi, clock and history

Enter your Wi-Fi in `secrets.h` (5.8), upload, and open the Serial Monitor. At the start you should see:

```
CO2 monitor starting
History: formatting the data partition (first start)
History: 8 KB of 1536 KB used
Wi-Fi: connecting to "MyNetwork"
SHT45 found
...
Wi-Fi connected, web page at http://192.168.1.50/
History: clock set to 2026-10-28 17:05:12 UTC
```

The "formatting" line appears only at the very first start, right after the first upload, so you'll usually miss it in the Serial Monitor. Every later start shows only the "KB used" line. Then:

1. Wait until the next full 10 minutes have passed (for example 17:10:05) and type `status`. `m1` and `m5` each show one file and a newest value.
2. Type `csv m1 1`. There is one line per minute since the start, and the CO_2 values match the live readings.
3. After the next full hour (plus 5 s), type `csv h1 1`. The first hour shows fewer than 60 in the `minutes` column, because the monitor started during that hour.
4. **Power cut test:** unplug the monitor for about 20 minutes, then plug it back in. In `csv m1 1` the gap starts at the last save before unplugging (at most 10 minutes earlier) and ends at the restart. Nothing else is missing.

5. **Optional, no internet at start:** switch off the router, restart the monitor, wait a few minutes, then switch the router back on. `status` shows "Clock: not set yet" until the router is back. Once the clock is set, the minutes measured without it appear in `csv m1 1` with the correct times.

Step 9 – Web page

1. Open the address from the Serial Monitor (`status` shows it too) in a browser on a phone or PC in the same Wi-Fi. The live values appear within 5 s, and the status line says "Live".
2. The calendar shows a dot under every day with values. Pick one, and the graph shows that day. Move the mouse over the graph: the values match `csv m1 1` for that minute. The page shows local time, the Serial Monitor UTC.
3. Press *3 h* above the graphs: they show the last 3 hours, and a minute later the range has moved on by a minute. Press *Night 22–08*: the range ends the next morning. Switch off two of the values under *Show*, and only one graph is left.
4. Download the last 7 days with *Best available detail* and open the file in Excel. There is one row per minute, already split into columns.
5. Switch your phone or PC between light and dark mode. With *Auto* the page follows.

10 CO_2 calibration (SCD-41)

The SCD-41 is **calibrated at the factory** by Sensirion and works out of the box. Over time, mechanical stress, soldering, shipping and ageing can shift its zero point (the "offset"). Two mechanisms correct this:

	ASC – automatic self-calibration	FRC – forced recalibration
Who does it	The **sensor itself** (firmware inside the SCD-41)	**Your code** sends one command
Default	**On** from the factory	Only when you trigger it
Code needed	None. Our main sketch leaves it as it is.	The optional `scd41_calibration.ino` tool
How it works	Watches the readings over days. Assumes the **lowest** level seen in each period was fresh outdoor air (target 400 ppm) and slowly corrects the offset. First correction after ~**44 h** of operation, then every ~**156 h** (6.5 days).	You put the sensor in air of known CO_2 (outdoors ≈ 425 ppm), let it run ≥ 3 min, and tell it "this is 425 ppm". The correction is applied immediately.
Requirement	The sensor sees fresh air (~400 ppm) **at least once per week of operating time**, while running **≥ 4 hours in one go**.	Stable, known reference air, and the sensor running in its normal mode for ≥ 3 minutes beforehand.
Use when	Normal homes and offices that get aired regularly.	Readings are clearly off in fresh air, or the room never gets fresh air (then also switch ASC off).

Your questions, answered

Is the CO_2 sensor calibrated by the code or by the sensor itself?

Both **Normally the sensor does it itself.** ASC runs inside the SCD-41 and is switched on at the factory, so no code is needed. Our main program doesn't touch the calibration; it only sends the altitude at every start. **FRC** is triggered by code when you know the true CO_2 level. The separate `scd41_calibration.ino` sketch does exactly that.

How should I start testing – open the door and let fresh air in, or just plug it in?

Just plug it in No fresh air is needed to start. Give it 5–10 minutes, do the breath test (CO_2 must jump), then do the fresh-air check from Testing, step 6. That check tells you whether the factory calibration is still good; it doesn't calibrate anything by itself.

Do I need fresh air every time I reconnect the board?

No The SCD-41 stores its calibration history (ASC and FRC) in its **own EEPROM automatically**, so it survives unplugging. You only need fresh air regularly *for ASC to keep working long term*: at least once per week of operating time, while the sensor runs for 4+ hours at a stretch. Normal airing of the room does that. Short sessions (unplugged after an hour) don't advance ASC, but they don't lose anything either.

What is stored where

Item	Stored	After unplugging
Factory calibration, ASC history, FRC correction	Sensor EEPROM (automatically)	**Kept**
Altitude, ASC on/off, temperature offset, ASC target	Sensor RAM; EEPROM only after `persistSettings()`	Lost, back to the saved values. Our code re-sends the altitude at every start, so there's no need to persist it.

The EEPROM is rated for only ~2000 writes. **Never call `persistSettings()` in `loop()`**, only once after a deliberate settings change. Our sketches never call it.

When ASC can go wrong

- **The room never gets fresh air** (always occupied, never aired). ASC then treats e.g. 700 ppm as "400" and pulls all readings too low. Switch ASC off with `setAutomaticSelfCalibrationEnabled(0)` + `persistSettings()` and use FRC every few months instead.
- **Target 400 vs real outdoor air:** ASC assumes 400 ppm, but outdoor air today is ~420–430 ppm. ASC can therefore read ~25 ppm low. That is well inside the sensor's accuracy, so it's fine for step 1. It could later be changed with `setAutomaticSelfCalibrationTarget()`.

FRC procedure (only if needed)

1. Upload `firmware/scd41_calibration/scd41_calibration.ino` (set `SENSOR_ALTITUDE_M` the same as in the main sketch).
2. Take the complete setup **outdoors** on a power bank or laptop: shade, away from roads, chimneys and people. Don't do it at an open window indoors; indoor air near a window is often 450–600 ppm.
3. Wait. The display counts down 5 minutes and shows the CO_2 value and the **spread** (max − min) of the last minute. A spread under ~20 ppm means stable air.
4. When it shows "press button", press the button (or send `c` in the Serial Monitor).
5. The display shows "Done! corr ±xx ppm". The new reading should now be close to 425 ppm. The correction is stored in the sensor automatically.
6. Upload `co2_monitor.ino` again.

The target is `FRC_TARGET_PPM = 425` (global outdoor background in 2026 is about 420–430 ppm). A wrong reference makes the sensor worse, not better, so only run FRC in genuinely fresh air.

11 Hardware notes

ESP32-C6-DevKitC-1-N8

- **Chip:** ESP32-C6 (in the ESP32-C6-WROOM-1-N8 module). 32-bit RISC-V main core up to 160 MHz plus a 20 MHz low-power core, 512 KB SRAM, **8 MB flash**.
- **Radio:** Wi-Fi 6 (2.4 GHz only), Bluetooth 5 LE, IEEE 802.15.4 (Zigbee / Thread / Matter). Not used yet. Keep the antenna end free of metal.
- **3.3 V logic, GPIOs not 5 V tolerant.** The 5V pin carries USB voltage. Never connect it to a GPIO or to the sensors' bus.
- **Two USB-C ports:** "USB" = native USB (Serial/JTAG, GPIO12/13), "UART" = USB-to-serial bridge (GPIO16/17). Either one powers and programs the board.
- **On board:** BOOT button (GPIO9), RESET button, addressable RGB LED (GPIO8), power LED. Jumper **J5** (2-pin header next to the "3V3" marking) carries the 3.3 V supply to the ESP32 module. It is fitted at the factory and **must stay fitted**; it is only removed to measure the module's current with an ammeter. Without it, the USB chip still shows up on the PC, but the ESP32 itself has no power.
- **Strapping pins** GPIO4, 5, 8, 9, 15: don't connect buttons or pull-downs to them.
- **Breadboard fit:** 25.4 × 51.8 mm, pin rows 22.86 mm (9 holes) apart. In columns b and i it leaves one free hole per pin on each side. The PCB extends ~12 mm past pin 16 (USB end) and the module overhangs ~8 mm past pin 1 (antenna end). With pins in rows 4–19, the USB ports end flush with the breadboard and the antenna covers rows 20–22, which leaves rows 23–30 free for the button and the bus.

Adafruit SCD-41 (Sensirion SCD41)

- **Principle:** photoacoustic NDIR. It measures real CO_2, unlike cheap "eCO_2" gas sensors that only estimate it.
- **Range / accuracy:** 400–5000 ppm specified at ±(40 ppm + 5 % of reading); outputs up to 40 000 ppm. (The SCD-40 is only specified to 2000 ppm.)
- **Modes:** periodic (every 5 s, ~17 mA; used here), low-power periodic (every 30 s), single shot (SCD-41 only, ~0.5 mA at one reading per 5 min; useful for battery operation later).
- **I^2C address 0x62** (fixed). 3.3–5 V supply; the breakout has a regulator and level shifting. A clean, stable supply matters for accuracy.
- **Built-in T/RH:** the SCD-41 always measures temperature and humidity; it needs them internally to compensate the CO_2 value, and this cannot be switched off. Every reading returns all three values. The code reads them and simply ignores T/RH, because the sensor warms itself slightly and the SHT45 is more accurate. Ignoring them has **no effect on CO_2 accuracy**, and neither does its temperature offset setting (it only shifts the reported T/RH). The SCD-41 cannot take temperature or humidity from the SHT45.
- **Air pressure:** this *does* affect CO_2 accuracy, so set your altitude in the code. Weather changes air pressure by a few percent; a pressure sensor added later could feed live values with `setAmbientPressure()`, which works during periodic measurement.
- **Handling:** the weak spot is the **white membrane on top of the metal cap**. It keeps dust and water out and lets CO_2 in. Sensirion: don't remove, damage, imprint or cover it. So don't touch or press it, and don't tape over it, clean it, or blow compressed air at it. The sensor is an optical system: avoid strong force on the cap, especially sideways, and drops onto hard floors. ESD rating is only 2 kV (HBM), so touch something metal first. Chemical fumes don't affect the CO_2 measurement the way they affect the SHT45's humidity reading. Keep dust, direct sunlight and strong drafts off it. Leave space for air to reach it. If it ever reads clearly wrong in fresh air after a knock, a forced recalibration fixes the offset (section 10). Calibration persists in its EEPROM.

Sensor choice: SCD-41 vs SCD-30

	SCD-41 (used)	SCD-30
Principle	Photoacoustic NDIR	Dual-channel NDIR (reference channel for long-term stability)
Specified range	400–5000 ppm	400–10 000 ppm
Accuracy	±(40 ppm + 5 %) → ±90 ppm at 1000 ppm	±(30 ppm + 3 %) → ±60 ppm at 1000 ppm
Sensor size	10.1 × 10.1 × 6.5 mm	35 × 23 × 7 mm
Power	~17 mA (every 5 s); single-shot mode ~0.5 mA (every 5 min)	~19 mA (every 2 s); no low-power single-shot mode
I^2C	Standard, up to 400 kHz	Max 100 kHz, needs clock stretching (up to 150 ms), a known source of trouble with some microcontrollers
Self-calibration	Fresh air once per week of operation, runs of ≥ 4 h	7 days continuous operation, ≥ 1 h fresh air per day
Adafruit breakout	STEMMA QT, 0x62	STEMMA QT, 0x61; $58.95 and often out of stock

Conclusion: for a room monitor the SCD-41 is the better fit. It is small, has trouble-free I^2C, and offers a low-power mode for a later battery version, and indoor air stays well inside 400–5000 ppm. The SCD-30 wins on paper accuracy, range and long-term stability. Choose it for mains-powered builds that may exceed 5000 ppm (greenhouse, grow tent, fermentation room) or where the best absolute accuracy matters more than size and power.

Adafruit SHT45 (Sensirion SHT45)

- **Accuracy:** ±0.1 °C typical (5–60 °C), ±1.0 %RH typical (20–70 %RH). Range −40…125 °C, 0…100 %RH.
- **I^2C address 0x44** (fixed). 3–5 V supply; the breakout has a regulator, level shifting and pull-ups.
- **Built-in heater:** for drying the sensor after condensation. While it runs, temperature readings are useless, so the code keeps it **off**.
- **Placement is everything:** the ESP32, the SCD-41, the display and your hand all give off heat. Mount the SHT45 at the end of the cable, away from warm parts and out of direct sunlight. Airflow improves response time.

Sensor handling: what can harm them

Source	Effect	What to do
Cleaning sprays (ammonia, bleach, alcohol), solvents (acetone / nail polish remover, paint, varnish), glue, silicone or epoxy while curing	**Permanent damage possible.** The SHT45's humidity reading drifts, in most cases for good. Sensirion names acetone, ethanol, isopropanol, toluene, ammonia and acids.	Don't spray or wipe near the sensors. Take them out of the room while painting or gluing, and air the room until the smell is gone.
Aerosols (hairspray, deodorant, spray paint), cooking grease, smoke	Coat the sensor openings.	Not in the kitchen or bathroom, no spraying nearby.
Water: spills, condensation (bathroom steam, cold window sill)	A short circuit can kill the boards. The SCD-41 is only rated for non-condensing air.	Keep everything dry. Unplug at once after a spill.
Static electricity	Can destroy the chips.	Touch something metal first (e.g. a radiator), hold the boards by their edges.
Pressing, dropping, pulling on cables	Broken connectors or solder joints.	Don't press on or poke into the SCD-41's sensor. Pull STEMMA plugs by the plug.

Vibration, loud bass (on top of a subwoofer or speaker)	No damage. Can make the SCD-41's readings noisier; Sensirion recommends keeping it decoupled from vibration.	Put it on a shelf or desk, not on the speaker.
Magnets (speakers, subwoofers, monitors, phone cases)	**No effect.** Neither sensor uses or reacts to magnetic fields.	Nothing.
Heat: sun, radiator, monitor, laptop vent, the ESP32, your hand	No damage at room conditions, but temperature reads too high and humidity too low.	See placement below.

How likely is it? Sensirion's rule of thumb: air that people can breathe for a long time without harm won't contaminate the sensor. Normal cleaning, cooking or a candle across the room is fine. The real risks are high concentration and long exposure: the sensor right next to curing glue or silicone, painting in a closed room, wiping the sensor with alcohol, or storing it in a box with solvents. Chemicals only affect the **humidity** reading; temperature isn't affected.

If the SHT45 does get contaminated

1. **Check it with the salt test.** Put a spoonful of table salt with a few drops of water (a damp paste, not dissolved) in a bottle cap. Put the cap and the SHT45 board in an airtight container or a zip bag, and run the STEMMA cable out through the seal. Make sure the salt doesn't touch the board. After 8–12 h at a steady room temperature the air inside is at **75 %RH**. The SHT45 should read 74–77 %RH. If it does, it's fine.
2. **Mild drift:** air the room well and leave the sensor running for a few days. Part of the contamination outgasses by itself.
3. **Heat it with its built-in heater.** In Sensirion's test, SHT4x sensors were exposed to saturated acetone for 20 h, far worse than anything at home. Heating with the on-chip heater (110 mW, 80 min, about 110 °C) brought them fully back within specification. The heater runs for at most 1 s per command and must stay below a 10 % duty cycle. On the Adafruit board the thicker PCB carries heat away, so the sensor gets less hot and the treatment has to run longer. Then let it rehydrate in the salt container (12 h at 75 %RH, the second half of Sensirion's reconditioning procedure).
4. **Oven baking** (Sensirion's full reconditioning: 10 h at 100–105 °C, then 12 h at 75 %RH) is **not recommended** for the breakout: the STEMMA QT connectors are only rated up to 85 °C, and a kitchen oven can't hold 100 °C accurately.
5. **Last resort:** correct a known, stable offset in software, or replace the board. It plugs in with the STEMMA cable, with no soldering.

Touching is fine if you hold the boards by their edges after touching something metal. A finger on the SHT45 shows up as a temperature and humidity rise that disappears within a minute. Don't put fingers on the sensor openings, because skin oil stays there.

Sensor placement

- In the room where you spend the most time (bedroom or office), at about **desk to head height (0.8–1.5 m)**.
- **About 1 m away from people's faces.** Directly in front of you on the desk, every breath reaches the sensor and the CO_2 value jumps around.
- Not near windows, doors, air vents, radiators, the stove, a humidifier, or in direct sunlight.
- Free air on all sides: not in a drawer, a closed shelf or behind the monitor.
- The SHT45 beside or below the ESP32, never above it (hot air rises), and at least 5–10 cm from the ESP32, the SCD-41 and the monitor.
- Air the room now and then: the SCD-41's self-calibration needs fresh-air levels at least once a week (section 10).

Joy-it 0.96" OLED (SBC-OLED01)

- 128×64 monochrome OLED, **SSD1306** controller, I²C. Address **0x3C** as delivered; **0x3D** after moving the resistor on the back.
- 3.3–5 V supply, 3.3 V logic. Pin order on this module: **GND, VCC, SCL, SDA** from left to right, header at the top edge. With the pins pointing out of the back, it lies flat when plugged into the breadboard.
- OLEDs **burn in** like OLED TVs and phones, but without their protection features: pixels that are lit for thousands of hours age faster and leave a faint ghost image. The fixed parts of the layout (the "CO2 ppm" label, the line, the °C sign) would show first. The 30 s timeout keeps the total on-time to minutes per day. If the display is ever kept on permanently, lower `OLED_BRIGHTNESS`, shift the image by a pixel every few minutes, or switch it off at night.
- **Brightness:** the SSD1306 contrast setting (command 0x81, 0–255) changes brightness only roughly 2× between 0 and 255. Darker than that needs extra settings (pre-charge period and V_{COMH} level).
- "Display off" (SSD1306 command 0xAE) turns the panel off and keeps its content; the controller then draws only a few µA.

Push button (4 legs)

- The four legs form two internally connected pairs; pressing the button joins the pairs. Using two **diagonally opposite** legs always works.
- No resistor is needed: the code enables the ESP32's internal pull-up (`INPUT_PULLUP`). Released = HIGH, pressed = LOW. The code debounces it (30 ms).

STEMMA QT cables

- 4-pin JST SH, 1.0 mm pitch, same as SparkFun Qwiic. Colours: **black = GND, red = V+, blue = SDA, yellow = SCL**.
- Each Adafruit board has two connectors, so modules can be chained. The connectors are small; pull them out by the plug, not the wires.

400-point breadboard

- 30 rows × 2 × 5 holes plus 4 power rails. Holes a–e of a row are connected, f–j are connected, not across the gap.
- Some breadboards have rails that are split in the middle. Check with a multimeter, or bridge the halves if yours is like that.

12 Troubleshooting

Symptom	Cause / fix
No COM port appears	Charge-only cable → use a data cable. Missing driver for the "UART" port → see the next row. Try the other USB-C port.
"CP2102N USB to UART Bridge Controller" under *Other devices*, Code 28	The CP210x driver isn't installed → install the Silicon Labs CP210x Universal Windows Driver (5.5). Or use the "USB" port with *USB CDC On Boot = Enabled*, which needs no driver.
Upload: "Failed to connect to ESP32-C6"	Hold BOOT, tap RESET, release BOOT, upload again. Select the right port. Try Upload Speed 115200. Check that the J5 jumper is fitted.
Serial Monitor shows nothing	Baud must be 115200. "USB CDC On Boot" must match the port (UART port → Disabled, USB port → Enabled). Press RESET.
Serial Monitor shows garbage	Wrong baud rate.

Scanner finds no devices	SDA/SCL swapped (GPIO23 = SDA, GPIO22 = SCL). No 3.3 V or GND on the bus rows (wires 1 and 2). OLED or STEMMA pins in the wrong rows.
Display found but stays dark	It timed out → press the button. Check `OLED_ADDRESS` (0x3C vs 0x3D, see scanner output).
Display text is upside down	Change `OLED_ROTATION` in the sketch (2 ↔ 0).
Display shows noise or a picture shifted by 2 pixels	The module has an SH1106 controller, not an SSD1306 (common on 1.3" modules). Switch the code to the *Adafruit SH110X* library.
Button does nothing	Button not across the gap, or both wires on the same internal leg pair → use diagonal legs f25 / e27. Check wire 5 (j8 = next to GPIO2).
"measuring…" never goes away after the warm-up	SCD-41 is not delivering values; check Serial for "SCD-41 not found" and the STEMMA cable.
CO_2 fluctuates by 10–30 ppm	Normal. People nearby, drafts and doors cause real changes too.
Temperature reads 1–3 °C too high	SHT45 too close to the ESP32, SCD-41, display, or in sunlight. Move it away.
Outdoor CO_2 clearly not ~420 ppm	Let ASC work for a week with regular airing, or run the FRC tool outdoors.
Board doesn't start after adding hardware	Something is connected to a strapping pin (GPIO4/5/8/9/15). Move it to a free GPIO.
Compile error about `SensirionI2cScd4x`	Old SCD4x library (0.x) installed → update to 1.x in the Library Manager.
Compile error "secrets.h is missing"	Copy `secrets_example.h` to `secrets.h` and enter your Wi-Fi (5.8).
Serial: `Wi-Fi: enter your network in secrets.h`	`secrets.h` still has the example name. Enter your network and upload again. Without Wi-Fi there's no clock, so nothing is recorded.
"Wi-Fi: connecting…" but never "connected"	Wrong name or password (both are case-sensitive), a 5 GHz-only network, or the router is too far away.
`status` : "Clock: not set yet" although Wi-Fi is connected	The router has no internet or blocks time requests (UDP port 123). The values wait in RAM for up to 24 h. If your router offers time itself, enter its address as `NTP_SERVER_1` .
Serial: `History OFF: wrong partition scheme`	Tools menu: Flash Size **8MB**, Partition Scheme **8M with spiffs (3MB APP/1.5MB SPIFFS)** (5.4), then upload again. The history is normally still there.
Commands in the Serial Monitor do nothing	The line-ending box next to the input line is set to "No Line Ending". Choose "New Line".
Web page doesn't open	The phone or PC is in another network (guest Wi-Fi, mobile data). Or the address has changed: `status` shows the current one, and a fixed address in the router (8.2) stops it changing.
Web page: "No connection to the monitor"	The monitor is unplugged, restarting or out of Wi-Fi range. The page keeps trying every 5 s and continues by itself.
Web page: "waiting for the internet time"	Same cause as "Clock: not set yet" above. The graphs appear as soon as the clock is set.

13 Case (design v3)

> **Approved design, 25 Oct 2026.** The drawings use datasheet sizes and the Berrybase breadboard (83 × 55 mm). The printable model follows once the measurements at the end of this section are in.

The design

- **Side by side, about 105 × 86 × 43 mm** including feet. The breadboard goes in unchanged; the USB cable leaves at the back (row-1 end).
- **Sensor chamber along the column-a side**, behind a **double wall with an air gap**. The ESP32's warmth rises out through the lid instead of reaching the sensors. BMP390, SCD-41 and SHT45 stand upright, sensors facing the side vents. The SHT45 is at the front and low: the point farthest from the ESP32 module, and more than 10 mm from the SCD-41, which warms itself slightly. Nothing can touch the SCD-41's membrane, and dust can't settle on it.
- **Vents:** the sensor chamber breathes through its outer side wall, its part of the front face and the floor (the case stands on feet). There are no vents in the top over the sensors, so no dust falls in. The electronics take cooling air in through the column-j side wall and let warm air out through lid vents directly above the ESP32 module. No fan.
- **Front face:** OLED behind a window. The light sensor (right-angle VEML7700) lies under the display behind a small window and measures the light falling on the display face. The RGB LED sits above the panel button, on the far side from the light sensor so it can't fool it.
- **Speaker** under the lid at the back with a grille in the lid: as far from the SCD-41 as possible, in a spot without wires. A **spare place** at the front takes one more STEMMA QT board later, for example an air-quality sensor.

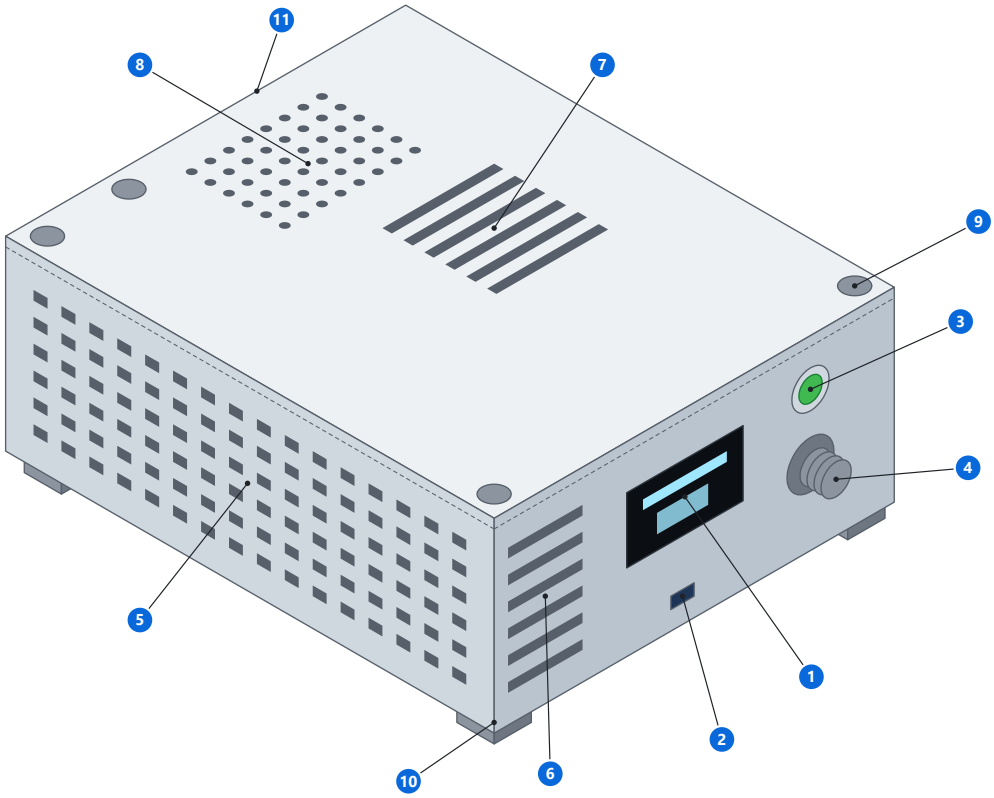


Figure 7 – Closed case seen from the front and the sensor-chamber side. (1) OLED window; (2) light-sensor window; (3) RGB LED; (4) panel button; (5) side vents and (6) front vents of the sensor chamber; (7) lid vents above the ESP32 module; (8) speaker grille; (9) lid screws, 4 × M3; (10) feet, with floor vents under the chamber between them; (11) back with the USB-C ports (Figure 10).

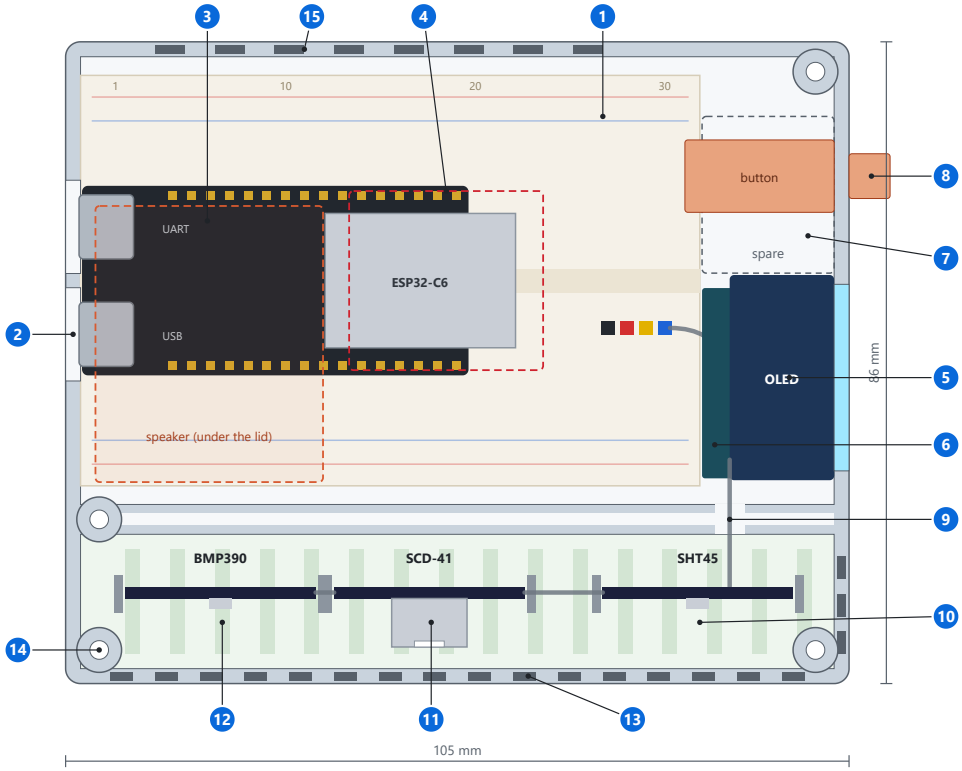


Figure 8 – Top view, lid removed, same orientation as Figure 3 (back with USB on the left, front on the right). (1) breadboard, unchanged; (2) ESP32 with both USB-C cut-outs; (3) speaker, hanging under the lid; (4) lid vents above the module (dashed red); (5) OLED behind the front window; (6) light sensor lying under the OLED; (7) spare place; (8) panel button; (9) sealed cable slot in the double wall; (10) SHT45, (11) SCD-41 and (12) BMP390 standing upright, sensors facing the outer wall; (13) side and floor vents of the chamber; (14) lid screw bosses; (15) intake vents for the electronics.

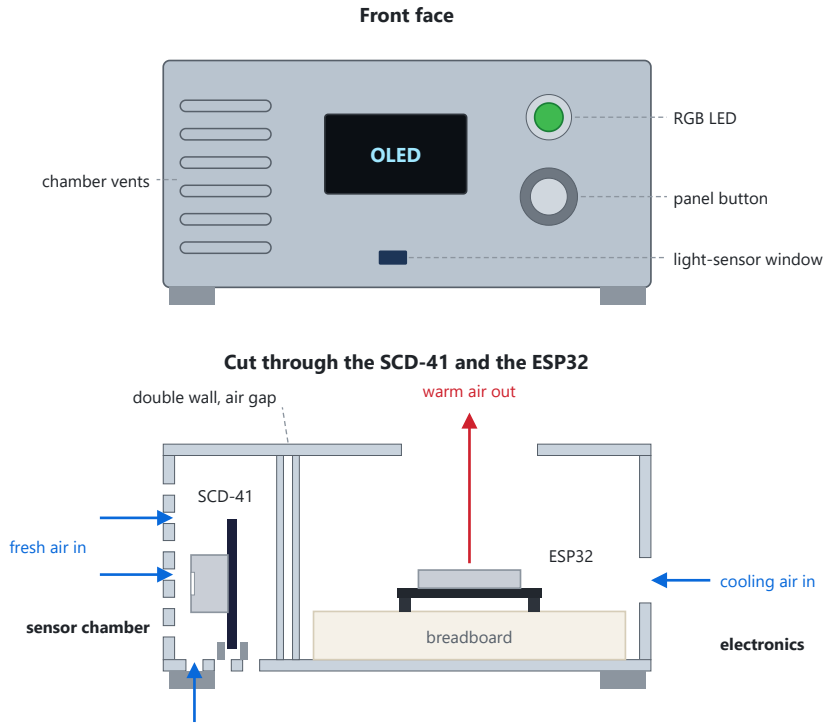


Figure 9 – Top: front face. Bottom: cut across the case through the SCD-41 and the ESP32, seen from the front. Blue: fresh and cooling air. Red: warm air from the ESP32 leaves through the lid, while the double wall keeps it out of the sensor chamber.

Printed parts

#	Part	Printed
1	**Base:** floor, walls, double wall, breadboard pocket, sensor slots, OLED bracket, light-sensor holder, screw bosses, vents, USB cut-outs, feet	upright
2	**Lid** with vents, speaker grille and speaker holder	upside down
3	**3 sensor clips** that hold the upright boards at the top	flat

All parts print without supports.

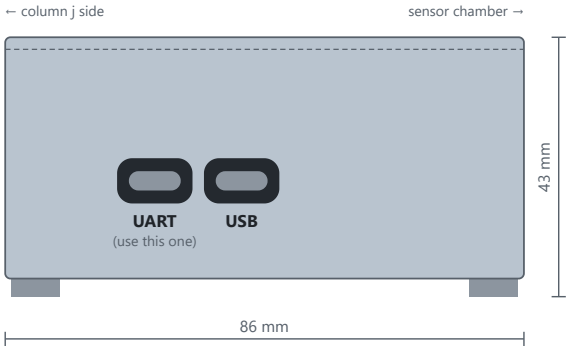


Figure 10 – Back, seen from outside. Both USB-C ports get a cut-out big enough for the plug's moulding; normally you use the "UART" one.

Parts to buy for the case

Part	For	Notes
Adafruit **STEMMA Speaker** (3885)	alarm beeps	37 × 30.5 × 7.2 mm, class-D amplifier, 3–5 V, 3-pin JST PH input
Adafruit **BMP390** (4816, also sold as BMP390L)	air pressure; feeds the SCD-41's pressure compensation	STEMMA QT, I²C 0x77
Adafruit **VEML7700 right angle** (5378)	ambient light; dims display and LED at night	STEMMA QT, I²C 0x10, sees light along the board
Panel push button, Berrybase 10131	replaces the breadboard button	M7 × 0.75 thread, 27.5 mm long
5 mm LED holder, 3 × 220 Ω resistors	your 4-leg RGB LED in the front wall	find out first whether the LED is common anode or common cathode
4 female–male jumper wires	OLED to rows 27–30	–
JST PH 3-pin to male-pin cable	speaker to the breadboard	–
2 STEMMA QT cables, 50–100 mm	BMP390 and VEML7700	–
4 × M3 × 10 screws	lid	self-tapping into the base
Filter fleece (optional)	behind the chamber vents	keeps dust and splashes out, lets gases through

Planned pins

Signal	GPIO	Breadboard	Note
Panel button	GPIO2	row 25 (as now)	replaces the breadboard button; other contact to GND
RGB LED red / green / blue	GPIO18 / 19 / 20	a10 / a11 / a12	each through a 220 Ω resistor; common leg to GND or 3.3 V depending on the LED type
Speaker signal	GPIO21	a13	PWM tones. Power 3.3 V, or 5 V from the 5V pin (free hole j6) for more volume; the 5 V goes to the speaker only.
OLED, BMP390, VEML7700	I²C (23 / 22)	rows 27–30 / STEMMA chain	new addresses 0x77 and 0x10 don't clash with 0x3C, 0x44, 0x62

GPIO18–21 are free, not strapping pins, and have free holes next to them in column a.

Design rules behind it

- Sensirion's design-in rules: sensors close to large openings with little dead air around them, decoupled from heat sources (the ESP32 module and its voltage regulator, both in the middle of the board), the temperature sensor low because warm air rises.
- The SHT45 sits at least 10 mm from the SCD-41 and never above it.
- The speaker is far from the SCD-41, which measures acoustically and doesn't like vibration.
- Walls, floor and lid 2 mm; double wall 2 × 1.2 mm with a 1.6 mm air gap; vent slots 2–3 mm wide (air gets in, fingers don't).

Needed for the printable model

1. **Breadboard:** height including the adhesive layer, and how far the side latches stick out.
2. **Tallest wire or cable:** press the jumper wires and cable pins down as flat as they go, then measure the highest point above the breadboard surface.
3. **Panel button:** usable thread length, and the nut and washer you got with it.
4. **OLED:** where the lit area is, the mounting holes (diameter and spacing) and how far the header pins stick out. A photo from straight above with a ruler next to it is enough.
5. **LED holder:** which one, and the hole size it needs.
6. **STEMMA QT cables** you have and their lengths.
7. **3D printer:** bed size, material (PLA or PETG) and nozzle size.
8. **Screws:** M3 available? Otherwise the lid gets snap hooks.

14 Change log

Version	Date	Changes
0.11	29 Oct 2026	Web page: the graphs are now right below the live values, with a heading that names the time shown. New buttons *Last 1 / 3 / 6 / 12 / 24 h* above them; the range moves along with the time (they replace *Last 24 h*). On a phone the day box now comes before the calendar. New screenshot (Figure 6), updated 8.3, 8.5 and test step 9. Tested on the board, on a PC and at phone width.
0.10	29 Oct 2026	Step 3: web page served by the monitor (new section 8, files `web.h`, `web.cpp`, `web/`). Live values, a calendar with marks on days that have values, a time range of up to 24 h, graphs of CO_2, temperature and humidity that can be switched on and off, a CSV download between two dates, and Auto/Light/Dark themes. Fixed address via the router (8.2). Uses 60 KB more program flash; the history storage is unchanged. New test step 9 and troubleshooting rows. Sections 8–13 are now 9–14, and the case drawings are now Figures 7–10. Tested on the board and in the browser, on a PC and at phone width.
0.9	29 Oct 2026	30 s warm-up after start-up (new setting `WARMUP_TIME_MS`): the sensors measure, but no value is shown, printed on Serial or saved in the history, and the readings from this time are thrown away. The display counts down meanwhile, then stays on for 30 s with the first values. `status` shows the seconds left. Updated test steps 3 and 4 and the troubleshooting row. Tested on the board: the warm-up ended exactly 30 s after the SCD-41 started, the first CO_2 value followed 3 s later.

0.8	28 Oct 2026	Step 2: Wi-Fi (name and password in `secrets.h`, which isn't in git) and internet time. New section 7: the values are stored in the flash with LittleFS as 1-minute values (7 days), 5-minute values (90 days) and hourly values with min/max (about 3½ years), saved every 10 minutes. New Serial Monitor commands `status`, `files`, `csv` and `save`. Warning about the partition scheme and "Erase All Flash" in 5.4, new setup step 5.8, test step 8 and troubleshooting rows. Sections 7–12 are now 8–13, and the case drawings are now Figures 6–9. Tested on the board: saving, restart and firmware upload keep the history.
0.7	25 Oct 2026	Section 11 now shows the approved case design v3: side by side, sensor chamber along the column-a side behind a double wall, display, light sensor, RGB LED and panel button on the front, speaker under the lid. New drawings, parts list, planned pins (RGB LED GPIO18–20, speaker GPIO21) and the measurements needed for the printable model.
0.6	25 Oct 2026	New section 11: concept for a 3D-printed case (flat box, display deck over the OLED, vented sensor chamber behind an inner wall), with four drawings, a parts list and the measurements needed for the printable files.
0.5	25 Oct 2026	New setting `OLED_BRIGHTNESS` (display brightness 0–255, default 207). RGB LED default brightness lowered to 5. More about OLED burn-in in section 9.
0.4.1	25 Oct 2026	Added sensor handling (what can harm the sensors, magnets, touching), placement advice, and how to check and restore a contaminated SHT45 (salt test, heater decontamination) to section 9. Expanded the SCD-41 handling note (membrane, mechanical force, ESD).
0.4	25 Oct 2026	The on-board RGB LED (GPIO8) now shows the CO_2 level, dimmed (`LED_BRIGHTNESS` = 10). Its colour changes smoothly from green (below 800 ppm) through yellow (1000) to red (2000 and above). It also turns off a colour left over from the BlinkRGB test at start-up.
0.3.1	25 Oct 2026	The "UART" USB port's CP2102N chip needed the Silicon Labs CP210x driver on this PC (Device Manager Code 28). Added the install steps (5.5) and a troubleshooting row. Clarified that the J5 jumper must stay fitted.
0.3	25 Oct 2026	Bus rows reordered to match the OLED's pins (row 27 GND, 28 3.3 V, 29 SCL, 30 SDA). OLED now lies flat in a27–a30, readable with the pins at the top; `OLED_ROTATION` back to 0. ESP32 wired straight to the bus rows (5 wires, power rails no longer used); button moved to rows 25–27 with its GND leg in the GND row.
0.2	25 Oct 2026	New breadboard layout: ESP32 in rows 4–19 with the USB ports flush with the board edge, bus moved to rows 27–30, button to rows 24–26. New `OLED_ROTATION` setting. Added SCD-41 vs SCD-30 comparison and notes on the SCD-41's internal T/RH.
0.1	25 Oct 2026	Initial setup: ESP32-C6 + SHT45 + SCD-41 + OLED on breadboard; button wakes display for 30 s; I^2C scanner; optional FRC calibration tool; this documentation.